

Autoregressive Modeling, Entropy, and Decoding

Mathematics of Deep Learning — Chapter 11

Yin Yang

Foundation Books Project

Roadmap

Tokens, Vocabularies, Logits, Probabilities

Autoregressive Factorization

Causal Decoder Outputs

Next-Token Training Objective

Entropy, Cross-Entropy, KL, Perplexity

From Training to Generation

Deterministic Decoding

Sampling-Based Decoding

Prompting and In-Context Conditioning

Evaluation, Calibration, Failure Modes

Summary

Tokens, Vocabularies, Logits, Probabilities

The Chapter's Running Question

Modern decoder language models repeatedly answer one question: *given the text so far, which token comes next?*

Running prompt:

the movie was not very

Toy vocabulary:

$\mathcal{V} = \{\text{good, bad, funny, long, <end>}\}.$

The model's job is to assign five numbers $p_{\theta}(v \mid \text{the movie was not very})$ for $v \in \mathcal{V}$.

A language model predicts a distribution. Decoding chooses one path through that distribution.

From Logits to Probabilities

Causal decoder returns logits $z \in \mathbb{R}^m$ at each position. Softmax normalizes them over the vocabulary:

$$p_k = \text{softmax}(z)_k = \frac{\exp(z_k)}{\sum_{j=1}^m \exp(z_j)}.$$

Running example, $z = (2.0, 1.0, 0.0, -0.5, -1.0)$:

Token	Logit	Probability
good	2.0	0.612
bad	1.0	0.225
funny	0.0	0.083
long	-0.5	0.050
<end>	-1.0	0.030

Only logit *differences* matter — shifting every logit by a constant leaves softmax unchanged.

Token IDs, Embeddings, Output Projection

Token ID

$\iota : \mathcal{V} \rightarrow \{1, \dots, m\}$ is an *address*, not a measurement. The number 2 – 1 does not mean bad is one unit from good.

Output projection (unembedding)

Hidden $h_t \in \mathbb{R}^d$, weights $W_U \in \mathbb{R}^{m \times d}$, bias $b \in \mathbb{R}^m$:

$$z_t = W_U h_t + b, \quad z_{t,k} = w_{U,k}^\top h_t + b_k.$$

Scale: $m = 50,000$, $d = 4096 \Rightarrow W_U$ has 204,800,000 entries.

Vocabulary size is a model-design choice, not just preprocessing.

Autoregressive Factorization

The Chain Rule, Used Left-to-Right

An autoregressive model factorizes a sequence as

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}),$$

where $x_{<t} = (x_1, \dots, x_{t-1})$ is the prefix before position t .

Taking logs turns products into sums:

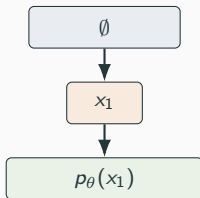
$$\log p_{\theta}(x_{1:T}) = \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}).$$

Concrete example with conditional probabilities 0.10, 0.40, 0.50:

$$p(x_{1:3}) = 0.10 \cdot 0.40 \cdot 0.50 = 0.020.$$

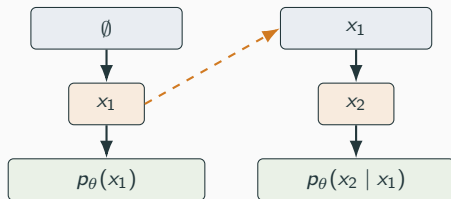
The factorization is a probability identity. The modeling choice is to estimate each conditional with a neural network.

The Prefix Ladder, Built Up



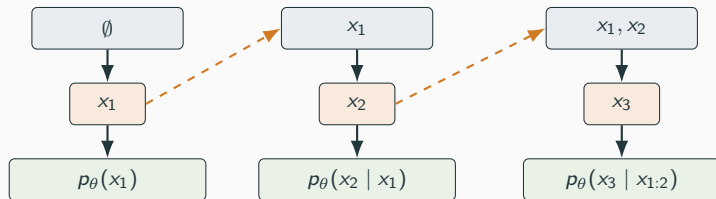
(1) First step conditions on the empty prefix.

The Prefix Ladder, Built Up



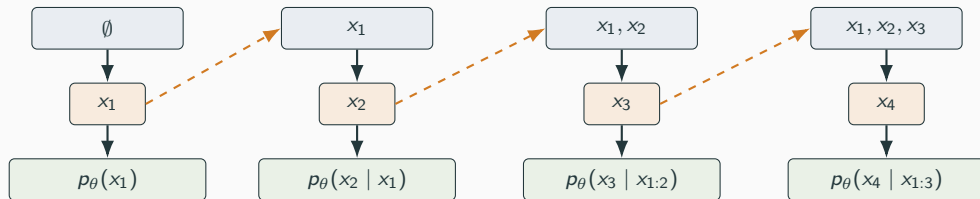
- (1) First step conditions on the empty prefix.
- (2) Each observed token extends the conditioning prefix.

The Prefix Ladder, Built Up



- (1) First step conditions on the empty prefix.
- (2) Each observed token extends the conditioning prefix.
- (3) The factor at rung t is the next-token probability.

The Prefix Ladder, Built Up



- (1) First step conditions on the empty prefix.
- (2) Each observed token extends the conditioning prefix.
- (3) The factor at rung t is the next-token probability.
- (4) The full sequence probability is the product of factors.

Each rung contributes one conditional factor.

End Tokens Normalize Over Finite Strings

Without a stop signal the chain can continue forever. Append a special `<end>` token to $\mathcal{V}$ and require generation to stop the first time it is emitted. Then

$$p_{\theta}(x_{1:T}, \text{<end>}) = \left(\prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}) \right) p_{\theta}(\text{<end>} \mid x_{1:T}).$$

If the process emits `<end>` with probability one eventually, these end-terminated probabilities sum to one over finite strings.

Otherwise some mass remains on infinite continuations.

An end token turns local next-token distributions into a proper distribution over finite strings.

Causal Decoder Outputs

Causal Decoder Interface

Input window of length $T \leq C$ (context length). After embedding lookup and causal transformer blocks:

$$H = \text{Decoder}_{\theta}(E[x_{1:T}] + P_{1:T}; M_{\text{causal}}) \in \mathbb{R}^{T \times d}.$$

Each row h_t depends only on tokens through position t — the prefix-function property of causal masking.

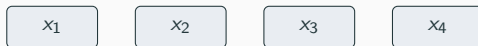
Output projection in a single matrix multiply:

$$Z = HW_U^{\top} + \mathbf{1}_T b^{\top} \in \mathbb{R}^{T \times m}.$$

Row t becomes the logit vector that predicts the shifted label x_{t+1} .

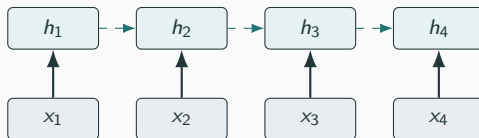
One forward pass produces T next-token distributions in parallel.

Shifted Outputs, Position by Position



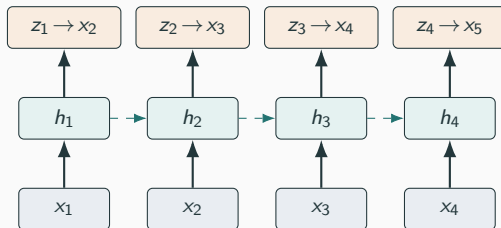
(1) Input tokens enter at each position.

Shifted Outputs, Position by Position



- (1) Input tokens enter at each position.
- (2) Causal blocks propagate information left-to-right only.

Shifted Outputs, Position by Position



- (1) Input tokens enter at each position.
- (2) Causal blocks propagate information left-to-right only.
- (3) Logits at position t are trained to predict x_{t+1} .

Causal masking is what makes row t legal for predicting the next token.

Next-Token Training Objective

Shifted Labels and Teacher Forcing

For the movie was not very good:

Prefix available	Next-token label
the	movie
the movie	was
the movie was	not
the movie was not	very
the movie was not very	good

Teacher forcing: during training the model conditions on the true previous tokens from the dataset, not on tokens it sampled itself. This makes the loss parallelizable across positions.

One forward pass produces losses for many labels at once.

Masked Average Next-Token Loss

Let $M_{i,t} \in \{0, 1\}$ indicate whether position (i, t) has a real shifted label. The training loss is

$$\mathcal{L}(\theta) = -\frac{1}{\sum_{i,t} M_{i,t}} \sum_{i,t} M_{i,t} \log p_{\theta}(x_{i,t+1} \mid x_{i,\leq t}).$$

Example: two padded sequences, token losses $(0.2, 0.7, 1.1, 0.0)$ and $(0.5, 0.4, 0.0, 0.0)$ with masks $(1, 1, 1, 0)$ and $(1, 1, 0, 0)$:

$$\mathcal{L} = \frac{0.2 + 0.7 + 1.1 + 0.5 + 0.4}{5} = 0.58.$$

Padded positions are ignored because their mask entries are zero.

Padding is storage. The mask defines which tokens count in the loss.

Softmax–Cross-Entropy Gradient

Position- t logits $z \in \mathbb{R}^m$, probabilities $p = \text{softmax}(z)$, target one-hot e_y where y is the shifted label index:

$$\ell(z, y) = -\log p_y, \quad \nabla_z \ell(z, y) = p - e_y.$$

Same gradient shape as ordinary multiclass classification — but with a *different* supervised label at every valid position.

Population decomposition (over random prefix $C = X_{<t}$):

$$\mathbb{E}_C \mathbb{E}_{Y \sim P(\cdot | C)} [-\log q_\theta(Y | C)] = \mathbb{E}_C H(P(\cdot | C)) + \mathbb{E}_C D_{\text{KL}}(P(\cdot | C) \| q_\theta(\cdot | C)).$$

Maximum likelihood targets the full conditional distribution, not just the most likely next token.

Entropy, Cross-Entropy, KL, Perplexity

Four Quantities, One Identity

Over a vocabulary distribution p :

$$H(p) = - \sum_k p_k \log p_k \quad (\text{entropy, in nats})$$

$$H(p, q) = - \sum_k p_k \log q_k \quad (\text{cross-entropy})$$

$$D_{\text{KL}}(p \| q) = \sum_k p_k \log \frac{p_k}{q_k} \quad (\text{KL divergence})$$

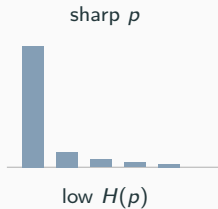
When $q_k > 0$ wherever $p_k > 0$,

$$H(p, q) = H(p) + D_{\text{KL}}(p \| q).$$

For one-hot targets, the token loss collapses to $-\log q_y$.

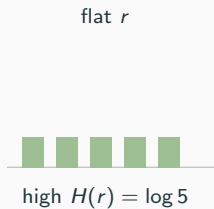
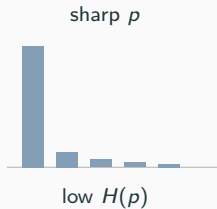
Cross-entropy is the irreducible entropy of the data plus a penalty for how far q is from p .

Sharp vs. Flat Distributions, Step by Step



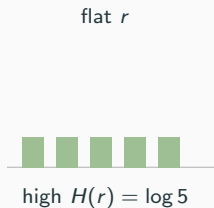
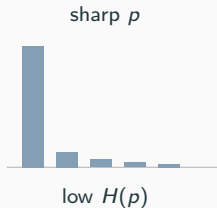
(1) A sharp distribution favors one token strongly.

Sharp vs. Flat Distributions, Step by Step



- (1) A sharp distribution favors one token strongly.
- (2) The uniform distribution maximizes entropy at $\log m$.

Sharp vs. Flat Distributions, Step by Step



average NLL L
 $\text{PPL} = \exp(L)$

- (1) A sharp distribution favors one token strongly.
- (2) The uniform distribution maximizes entropy at $\log m$.
- (3) Perplexity exponentiates the average NLL.

Perplexity scores prediction, not truth.

Perplexity in Numbers

Average token NLL $L \Rightarrow \text{PPL} = \exp(L)$ (or 2^L with $\log_2$).

L (nats)	$\text{PPL} = e^L$	Reading
0.7	≈ 2.01	like guessing among ≈ 2 tokens
1.5	≈ 4.48	like guessing among ≈ 4 tokens
3.0	≈ 20.1	much more uncertain

Caveats:

- Tokenization changes the unit. Word and subword perplexities are not directly comparable.
- Context length and preprocessing must match.
- A fluent false sentence can still receive low perplexity.

Lower perplexity means better next-token prediction under one fixed evaluation setup — nothing more.

From Training to Generation

The Generation Loop, Built Up

current prefix

$$x_{\leq t}$$

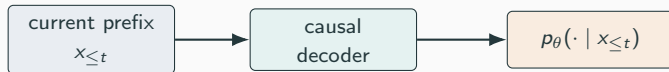
(1) Start from the current prefix.

The Generation Loop, Built Up



- (1) Start from the current prefix.
- (2) Decoder produces hidden states; output projection gives logits.

The Generation Loop, Built Up



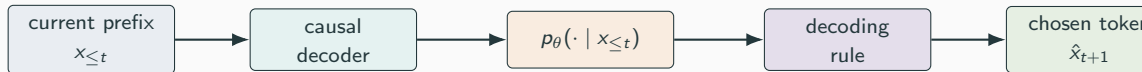
- (1) Start from the current prefix.
- (2) Decoder produces hidden states; output projection gives logits.
- (3) Softmax yields the next-token distribution.

The Generation Loop, Built Up



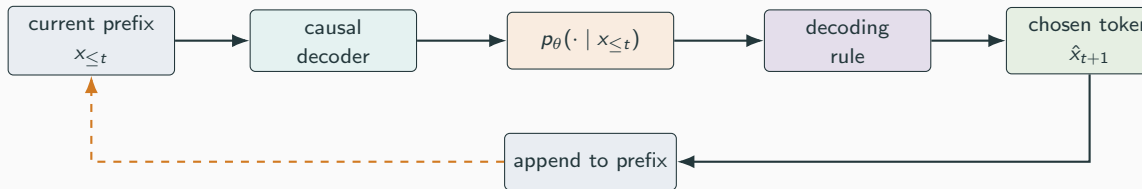
- (1) Start from the current prefix.
- (2) Decoder produces hidden states; output projection gives logits.
- (3) Softmax yields the next-token distribution.
- (4) A decoding rule picks (or samples) one token.

The Generation Loop, Built Up



- (1) Start from the current prefix.
- (2) Decoder produces hidden states; output projection gives logits.
- (3) Softmax yields the next-token distribution.
- (4) A decoding rule picks (or samples) one token.
- (5) The chosen token is the visible output.

The Generation Loop, Built Up



- (1) Start from the current prefix.
- (2) Decoder produces hidden states; output projection gives logits.
- (3) Softmax yields the next-token distribution.
- (4) A decoding rule picks (or samples) one token.
- (5) The chosen token is the visible output.
- (6) It is appended to the prefix and the loop repeats.

Training scores true text; generation feeds back chosen text.

Training vs. Generation, Side by Side

	Training	Generation
Prefix source	dataset (teacher forcing)	decoded so far (self)
Parallelism	all positions in parallel	one position at a time
Output	per-token loss	one chosen token
Randomness	none (fixed labels)	depends on decoding rule

Exposure bias: the gap between true prefixes seen at training and self-produced prefixes seen at generation. Early choices can make later choices easier, harder, more repetitive, or more surprising.

Evaluate generated text under the decoding rule you plan to use — not only by training loss.

Deterministic Decoding

Greedy and Beam Search

Greedy

$\hat{x}_{t+1} = \arg \max_{v \in \mathcal{V}} p_{\theta}(v \mid \hat{x}_{\leq t})$. Cheap, deterministic, locally best.

Beam search (*B*-best)

Keep $\mathcal{B}_t$ of size at most B , scoring partial sequences by

$$S(\hat{x}_{1:t}) = \sum_{s=1}^t \log p_{\theta}(\hat{x}_s \mid \hat{x}_{<s}).$$

Failure mode for greedy: a locally best first token can lead to a globally worse sequence.

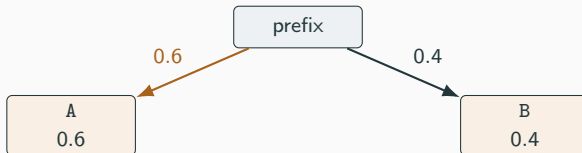
Beam search pays extra computation to keep alternatives alive.

Greedy Misses a Better Path, Step by Step

prefix

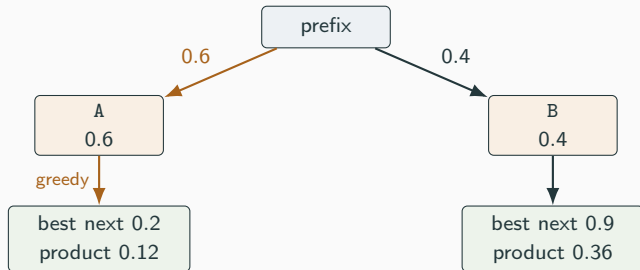
(1) A single prefix to extend.

Greedy Misses a Better Path, Step by Step



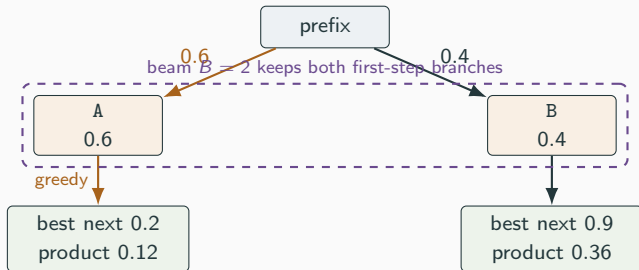
- (1) A single prefix to extend.
- (2) A wins step one ($0.6 > 0.4$).

Greedy Misses a Better Path, Step by Step



- (1) A single prefix to extend.
- (2) A wins step one ($0.6 > 0.4$).
- (3) But B's best continuation gives $0.4 \cdot 0.9 = 0.36 > 0.12$.

Greedy Misses a Better Path, Step by Step



- (1) A single prefix to extend.
- (2) A wins step one ($0.6 > 0.4$).
- (3) But B's best continuation gives $0.4 \cdot 0.9 = 0.36 > 0.12$.
- (4) A beam of width 2 keeps B alive to win at step two.

Greedy is the beam-search special case $B = 1$.

Length Normalization

Raw beam scores shrink as more probabilities are multiplied:

$$S(\hat{x}_{1:t}) = \sum_{s=1}^t \log p_{\theta}(\hat{x}_s \mid \hat{x}_{<s}) \xrightarrow{t \uparrow} -\infty.$$

Length-normalized variant:

$$S_{\text{avg}}(\hat{x}_{1:t}) = \frac{1}{t} \sum_{s=1}^t \log p_{\theta}(\hat{x}_s \mid \hat{x}_{<s}).$$

Without it, the search can prefer short outputs for a purely arithmetic reason. In open-ended generation, maximizing probability too aggressively can also produce dull or repetitive text.

High likelihood is the training objective. It is not always the best decoding objective.

Sampling-Based Decoding

Temperature-Scaled Softmax

Temperature $\tau > 0$ rescales logits before softmax:

$$p_k(\tau) = \frac{\exp(z_k/\tau)}{\sum_j \exp(z_j/\tau)}.$$

Running logits $z = (2, 1, 0, -0.5, -1)$:

τ	good	bad	funny	long	<end>	$H(p(\tau))$
0.5	0.860	0.116	0.016	0.006	0.002	0.488
1.0	0.612	0.225	0.083	0.050	0.030	1.099
2.0	0.403	0.244	0.148	0.115	0.090	1.459

Low τ sharpens. High τ flattens. $\tau \rightarrow 0$ recovers greedy.

Temperature controls how concentrated the next-token distribution is.

Top- k and Top- p (Nucleus)

Top- k

Keep the k highest-probability tokens, zero out the rest, renormalize, sample.

Top- p (nucleus)

Sort tokens by decreasing probability. Keep the smallest set whose cumulative mass first reaches p . Renormalize, sample.

Token	Prob.	Cum.	Top-2	Top- $p=0.90$
good	0.61	0.61	keep	keep
bad	0.23	0.84	keep	keep
funny	0.08	0.92	drop	keep
long	0.05	0.97	drop	drop
<end>	0.03	1.00	drop	drop

Top- p keeps a variable-size set; top- k keeps a fixed-size set.

Decoding Rules at a Glance

Rule	Candidate set on example	Consequence
Greedy	{good}	deterministic, narrow
Temperature	all five tokens	reweights, same support
Top-2	{good, bad}	fixed support
Top- $p=0.90$	{good, bad, funny}	adaptive support
Beam $B=4$	several partial sequences	deterministic, multi-path

Sampling settings are applied *after* training. They change how probability mass is converted into generated text without touching any parameter.

Same trained model, different decoding rule, different observable behavior.

Prompting and In-Context Conditioning

Prompting is Conditioning

If c is the prompt and $y_{1:U}$ is a possible answer,

$$p_{\theta}(y_{1:U} \mid c) = \prod_{j=1}^U p_{\theta}(y_j \mid c, y_{1:j-1}).$$

Two prompts c, c' can assign different probabilities to the same answer:

$$\Delta(c, c'; y_{1:U}) = \log p_{\theta}(y_{1:U} \mid c') - \log p_{\theta}(y_{1:U} \mid c).$$

The parameter θ is identical in both terms.

Changing the prompt changes the conditioning prefix, which changes every factor in the product — without any training step.

A Two-Answer Logit Calculation

Suppose the next token must be positive or negative.

Vague prompt c produces logits $(0.2, 0.1)$:

$$p_{\theta}(\text{positive} \mid c) = \frac{e^{0.2}}{e^{0.2} + e^{0.1}} \approx 0.525.$$

After adding demonstrations, the same parameters produce $(1.4, -0.6)$:

$$p_{\theta}(\text{positive} \mid c') = \frac{e^{1.4}}{e^{1.4} + e^{-0.6}} \approx 0.881.$$

In-context examples are tokens in the context window, not gradient updates. They still strongly affect the next-token distribution because the model conditions on them.

Prompting changes conditioning, not parameters.

Instruction, Examples, Query

Decompose the prompt as $c = \text{encode}(I, E, q)$:

- I : instruction — task rule or output format.
- E : optional in-context examples.
- q : query whose answer is to be generated.

All three are part of the conditioning context.

A vague prompt leaves many likely answer formats. An explicit instruction with examples can concentrate probability mass on the desired form.

Prompting is not the same as learning — imitating a pattern in a prompt is not proof of a reliable underlying concept.

Evaluation, Calibration, Failure Modes

Held-Out Validation Loss

Held-out token sequences $\mathcal{D}_{\text{val}}$ with masks $m_t^{(n)} \in \{0, 1\}$:

$$\hat{L}_{\text{val}}(\theta) = \frac{1}{N_{\text{val}}} \sum_n \sum_t m_t^{(n)} [-\log p_{\theta}(x_{t+1}^{(n)} \mid x_{1:t}^{(n)})],$$

with $N_{\text{val}} = \sum_n \sum_t m_t^{(n)}$.

Reported perplexity is $\exp(\hat{L}_{\text{val}})$ with natural logs.

Comparability conditions:

- Same tokenization across systems being compared.
- Same context length and preprocessing.
- Same set of included positions (mask).

Perplexity numbers are comparable only when the evaluation setup is.

Calibration vs. Accuracy

Let $\hat{Y} = \arg \max_v p_\theta(v \mid X_{1:t})$, $C = \max_v p_\theta(v \mid X_{1:t})$, $Z = \mathbf{1}\{X_{t+1} = \hat{Y}\}$. Exact calibration: $\mathbb{E}[Z \mid C] = C$ almost surely.

Expected calibration error with confidence bins $B_1, \dots, B_K$:

$$\text{ECE} = \sum_{k=1}^K \frac{|B_k|}{N} |\text{acc}(B_k) - \text{conf}(B_k)|.$$

A model can choose the right top token while being overconfident or underconfident. Calibration matters when probabilities decide whether to defer, ask, or pick a conservative decoding setting.

Calibration is different from accuracy — both are needed for trustworthy probabilistic outputs.

Failure Modes the Loss Does Not Catch

- **Distribution shift:** $q_{\text{deploy}} \neq q_{\text{val}}$. Validation perplexity estimates the loss on q_{val} , not on the deployment text.
- **Memorization:** reproduction of a training substring from $\mathcal{T}$ above a length or similarity threshold.
- **Data contamination:** an evaluation example e near-duplicates a training example t .
- **Hallucination:** a fluent claim a where an external verifier V returns $V(a) = 0$.

The mathematics has not failed — NLL is just measuring something narrower than “factual” or “useful.”

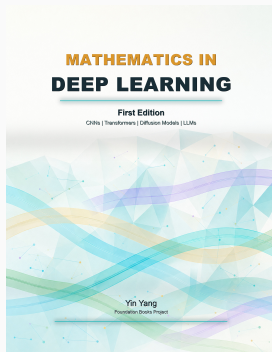
Pick a metric that matches the deployment risk; perplexity alone is rarely enough.

Summary

Chapter in One Slide

- **Tokens to probabilities:** ID is an address; logits are scores; softmax produces $p_{\theta}(\cdot \mid x_{\leq t})$.
- **Autoregressive factorization:** chain rule applied left-to-right; end token normalizes over finite strings.
- **Causal decoder:** one forward pass produces T next-token distributions thanks to causal masking.
- **Training:** masked average next-token NLL; gradient $p - e_y$ at each valid position; teacher forcing.
- **Entropy/CE/KL/PPL:** $H(p, q) = H(p) + D_{\text{KL}}(p \parallel q)$; $\text{PPL} = \exp(L)$ measures held-out prediction, not truth.
- **Decoding:** greedy, beam, temperature, top- k , top- p — same parameters, different generated text.
- **Prompting:** conditioning, not learning. Changes every factor in the product without touching θ .
- **Evaluation:** held-out loss + calibration + shift + memorization + hallucination, used together.

Next: scaling, fine-tuning, and alignment — where the same conditional distributions are reshaped by additional training signals.



Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/mathematics-in-deep-learning
Code	github.com/foundation-books/mathematics-in-deep-learning-code
Paperback	amazon.com/dp/BOH1ZZHDT2
Hardcover	amazon.com/dp/BOH1ZL11MJ
Kindle	amazon.com/dp/B0GX32M8SP
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.